



Course Details

Course Title: Digital Image Processing

Couse Code: CSE 438

Section: 02

Instructor

Dr. Ahmed Wasif Reza

Professor & Dean

Department of Computer Science and Engineering

Submitted To

Md. Adnan Morshed

Lecturer

Department of Computer Science and Engineering

Submitted By

Farhatun Nahar Priya

Student ID: 2023-1-60-202

Date of Submission: 25th September 2026.

1. Introduction

This lab applies basic digital image processing techniques for grayscale images. The tasks include perimeter detection, thresholding, connected component counting, Euclidean distance, arithmetic image operations and logical operations.

Question 1. Determine the perimeter of Figure 1 using 4 connected and 8 connected neighbourhoods.

Code:

```
from scipy.ndimage import binary_erosion

# Convert grayscale image to binary (0 or 1)
_,binary = cv2.threshold(fig1, 127, 1, cv2.THRESH_BINARY)

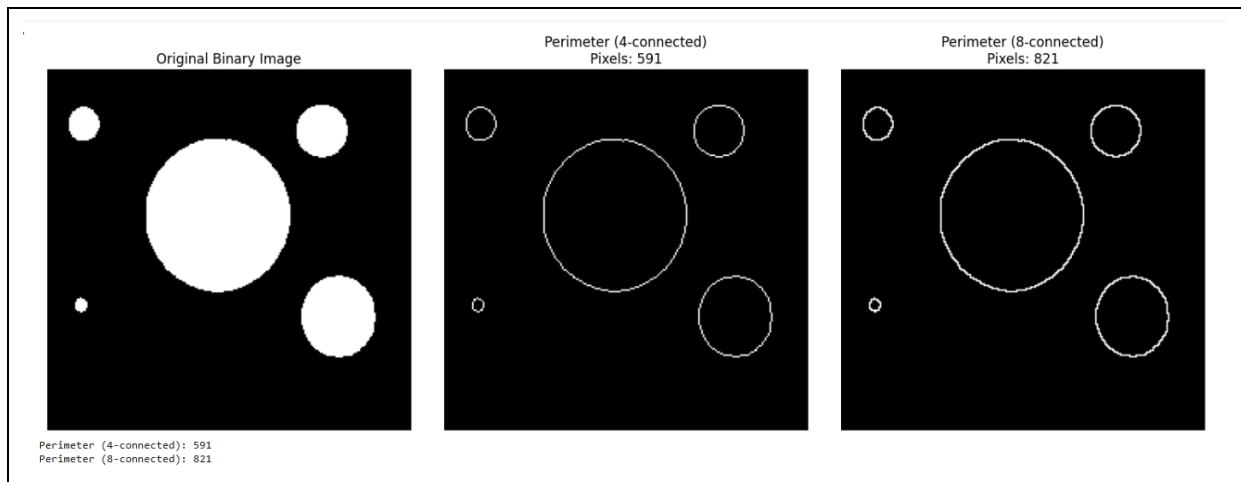
# Compute perimeter using structuring element
def compute_perimeter(binary_image, connectivity = 4):
    struct = np.array([[0,1,0],
                       [1,1,1],
                       [0,1,0]]) if connectivity == 4 else np.ones((3,3))
    eroded = binary_erosion(binary_image, structure = struct)
    perimeter = binary_image - eroded
    return perimeter, int(np.sum(perimeter))

#Compute both perimeter
perim4_img, perim4_count = compute_perimeter(binary, connectivity=4)
perim8_img, perim8_count = compute_perimeter(binary, connectivity=8)

# Plotting: original + two perimeter results
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(binary, cmap='gray')
axes[0].set_title("Original Binary Image")
axes[0].axis("off")
axes[1].imshow(perim4_img, cmap='gray')
axes[1].set_title(f"Perimeter (4-connected)\nPixels: {perim4_count}")
axes[1].axis("off")
axes[2].imshow(perim8_img, cmap='gray')
axes[2].set_title(f"Perimeter (8-connected)\nPixels: {perim8_count}")
axes[2].axis("off")
plt.tight_layout()
plt.show()
print(f"Perimeter (4-connected): {perim4_count}")
print(f"Perimeter (8-connected): {perim8_count}")
```

Output:



Explanation:

The image is eroded to find its boundary. In 4 connectivity, only the top, bottom, left and right neighbours are considered, so the result is 591 pixels. In 8 connectivity, diagonal neighbours are also considered, so more boundary pixels are found, giving 821 pixels.

Question 2. Create a binary image using a threshold.

Code:

```
from scipy.ndimage import binary_erosion
_,binary=cv2.threshold(fig2,127,1,cv2.THRESH_BINARY)
show_image(binary)
```

Output:



Explanation:

A threshold value of 127 is used to convert the grayscale image into a binary image. Pixels greater than 127 become white (1), while the other pixels become black (0). This makes the rice objects clearly separated from the dark background.

Question 3 Number of Objects Using Connectivity.

Code:

```
num_labels, labels = cv2.connectedComponents(binary)
print(f"Number of objects: {num_labels - 1}")
```

Output:

```
Number of objects: 89
```

Explanation:

The output shows 89 objects. Here, `cv2.connectedComponents()` scans the binary image and groups connected white pixels together, giving each group a unique label. It found a total of `num_labels` groups, but one of them (label 0) is always the background, not an actual object.

So we subtract 1 (num_labels - 1) to get the real object count. This means the binary image contains 89 separate connected white regions (objects), not counting the background.

Question 4. Find the Euclidean Distance between two points of the image.

Code:

```
pt1 = (10, 10)
pt2 = (50, 50)
dist = distance.euclidean(pt1, pt2)
print("Euclidean Distance between two points of the image:",dist)
```

Output:

```
Euclidean Distance between two points of the image: 56.568542494923804
```

Explanation:

The output gives a Euclidean distance of 56.568542494923804 between point (10,10) and point (50,50), which matches the formula $\sqrt{((50-10)^2 + (50-10)^2)} = \sqrt{3200} \approx 56.57$. This confirms the straight-line distance between the two points is approximately 56.57 units.

Q 5. Apply the following operations using Fig.1 and Fig.2:

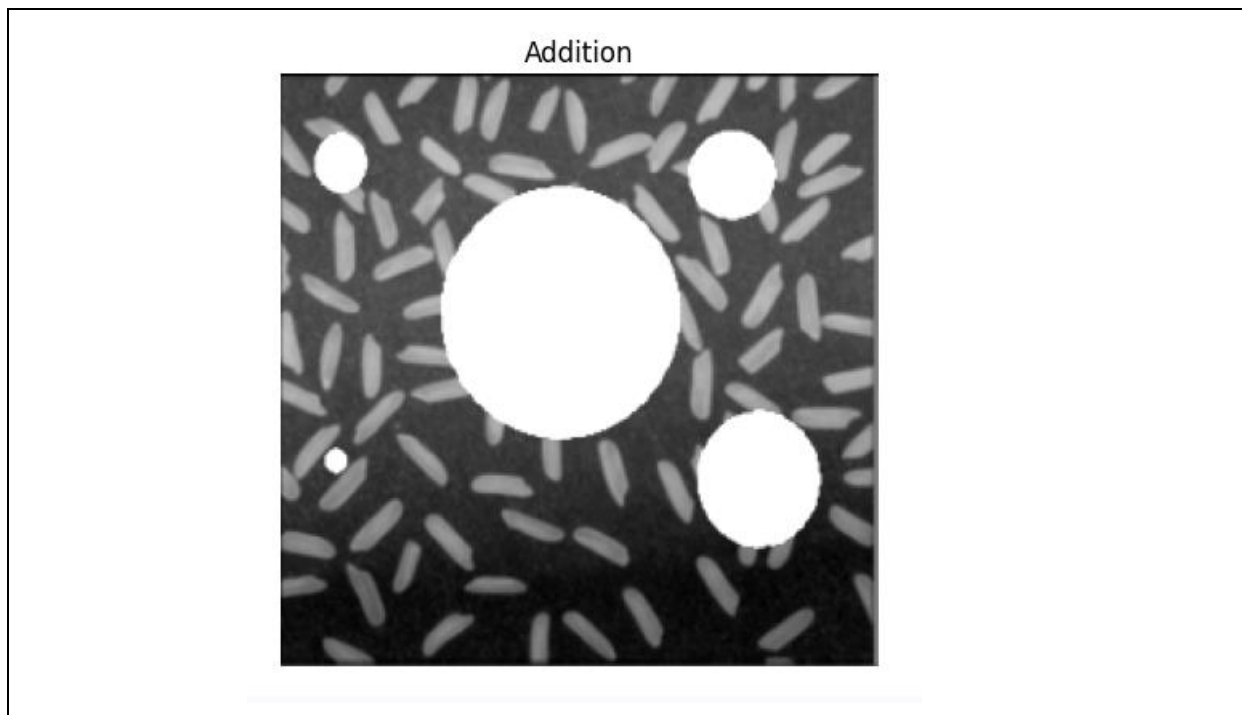
1. Addition
2. Subtraction
3. Multiplication
4. Division

1. Addition

Code:

```
addition = cv2.add(fig1, fig2)
plt.imshow(addition, cmap='gray')
plt.title("Addition")
plt.axis("off")
plt.show()
```

Output:



Explanation:

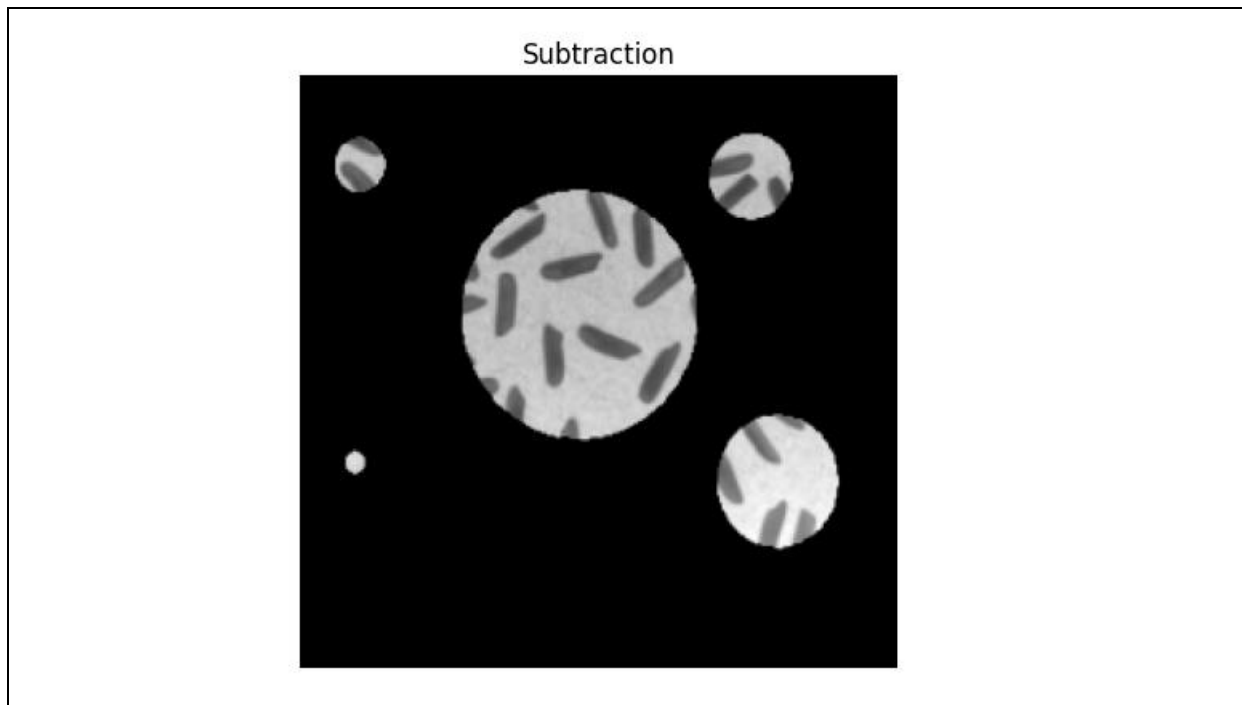
The output image shows the result of adding `fig1` and `fig2` using `cv2.add()`. The white circular shapes from `fig2` appear clearly on top of the grain-textured background of `fig1`, since pixel wise addition increases brightness where both images overlap. This confirms that `cv2.add()` successfully combined the two images while keeping pixel values within the valid range (0-255).

2. Subtraction

Code:

```
subtraction = cv2.subtract(fig1, fig2)
plt.imshow(subtraction, cmap='gray')
plt.title("Subtraction")
plt.axis("off")
plt.show()
```

Output:



Explanation:

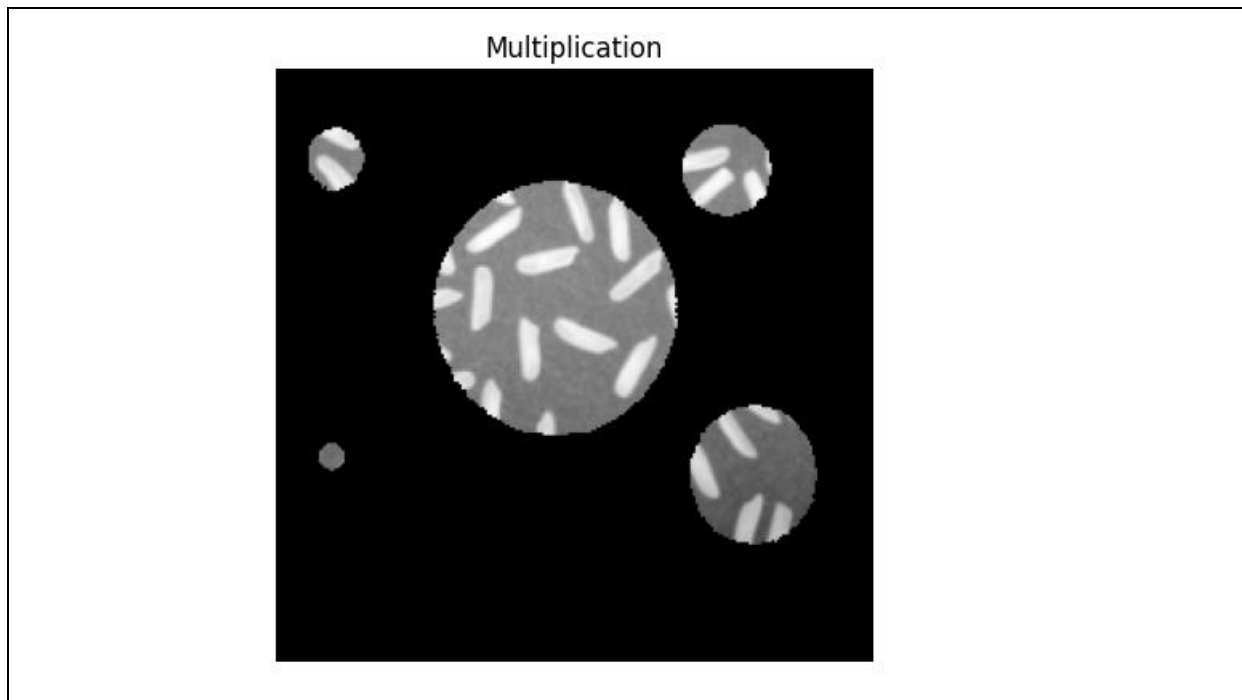
The output image shows the result of subtracting `fig2` from `fig1` using `cv2.subtract()`. The grain-textured background is mostly removed (turned black), while the circular regions from `fig2` remain visible, since pixel-wise subtraction highlights the differences between the two images. This confirms that `cv2.subtract()` correctly removed the common background and kept only the non-overlapping bright regions.

3. Multiplication

Code:

```
multiplication = cv2.bitwise_and(fig2, fig2, mask=fig1)
plt.imshow(multiplication, cmap='gray')
plt.title("Multiplication")
plt.axis("off")
plt.show()
```

Output:



Explanation:

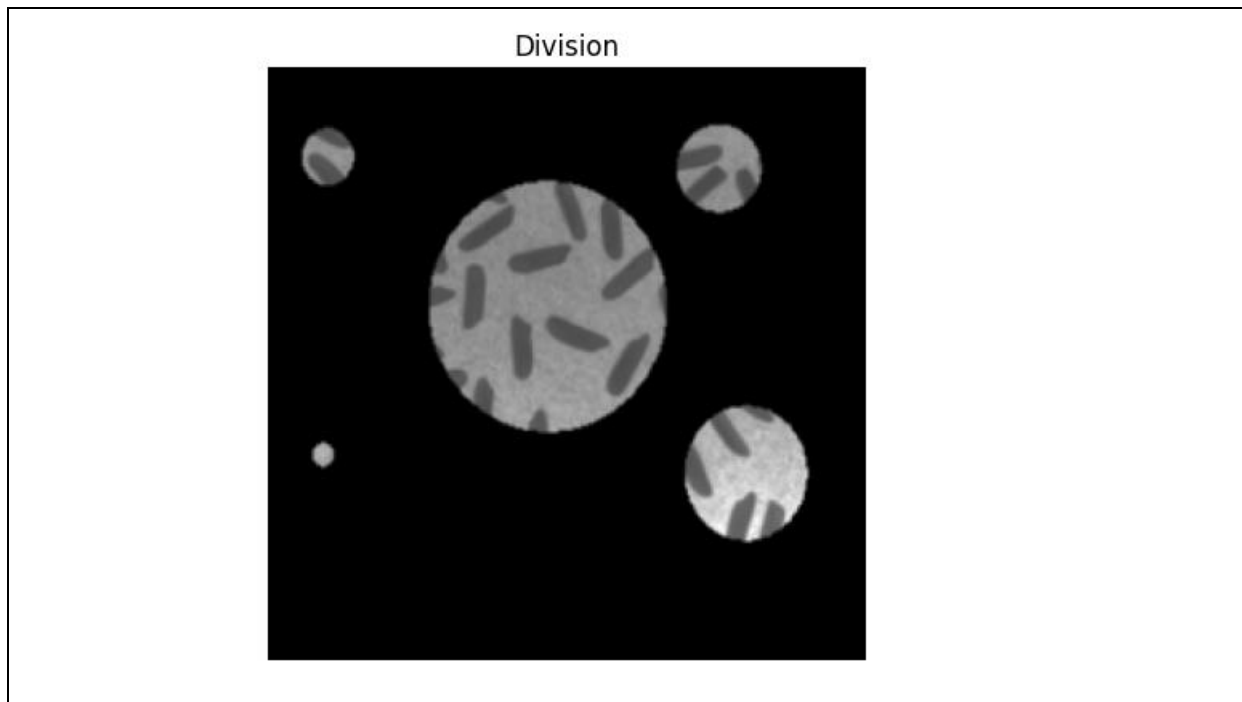
The output image shows the result of `cv2.bitwise_and(fig2, fig2, mask=fig1)`, which uses `fig1` as a mask to extract only the regions of `fig2` that fall inside `fig1`'s white areas. The output shows the grain textured background (from `fig2`) appearing only within the circular regions defined by `fig1`, while everything outside those circles remains black. This confirms that the multiplication (masking) operation successfully isolated the overlapping regions between the two images.

4. Division

Code:

```
fig1_float = fig1.astype(np.float32)
fig2_float = fig2.astype(np.float32) + 1
division = cv2.divide(fig1_float, fig2_float)
plt.imshow(division, cmap='gray')
plt.title("Division")
plt.axis("off")
plt.show()
```

Output:



Explanation:

The output image shows the result of dividing `fig1` by `fig2` using `cv2.divide()`, after converting both images to float type and adding 1 to `fig2` to avoid division by zero. The output looks visually similar to `fig1`, with the grain shapes and circular regions still visible, since dividing by values close to 1 keeps the pixel intensities nearly unchanged. This confirms that `cv2.divide()` performed pixel-wise division correctly while avoiding errors from zero-valued pixels.

Q 6. Apply the following operations using Fig.1 and Fig.2:

1. AND
2. OR
3. NOT

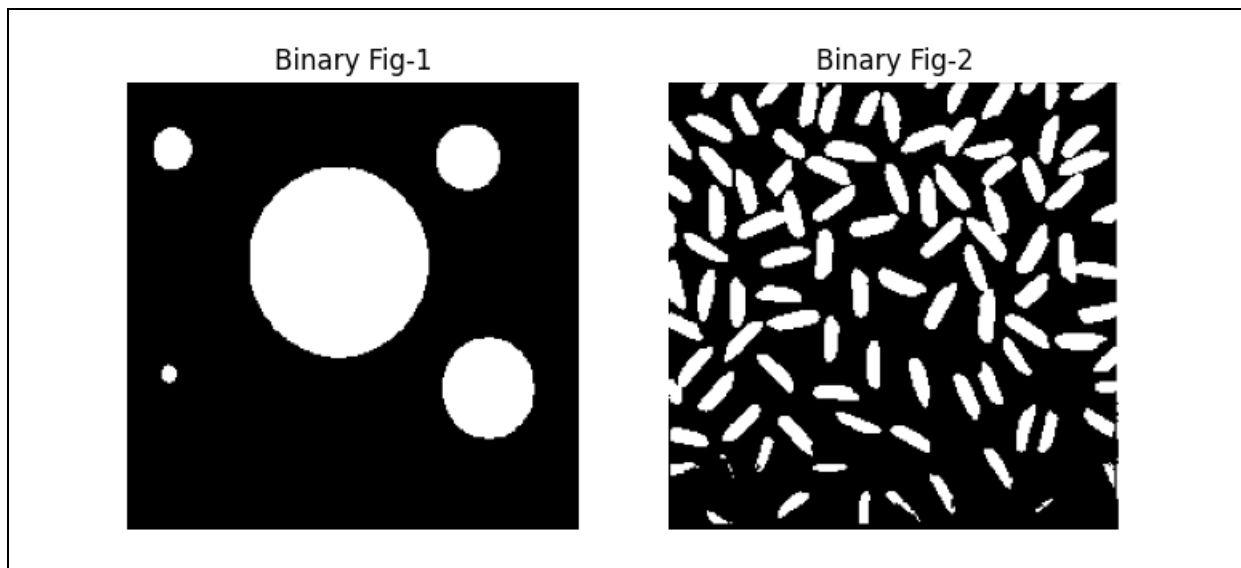
Code:

```
# Converting Images to Binary

_, fig1_bin = cv2.threshold(fig1, 127, 255, cv2.THRESH_BINARY)
_, fig2_bin = cv2.threshold(fig2, 127, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.imshow(fig1_bin, cmap='gray')
plt.title("Binary Fig-1")
plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(fig2_bin, cmap='gray')
plt.title("Binary Fig-2")
plt.axis("off")
plt.show()
```

Output:



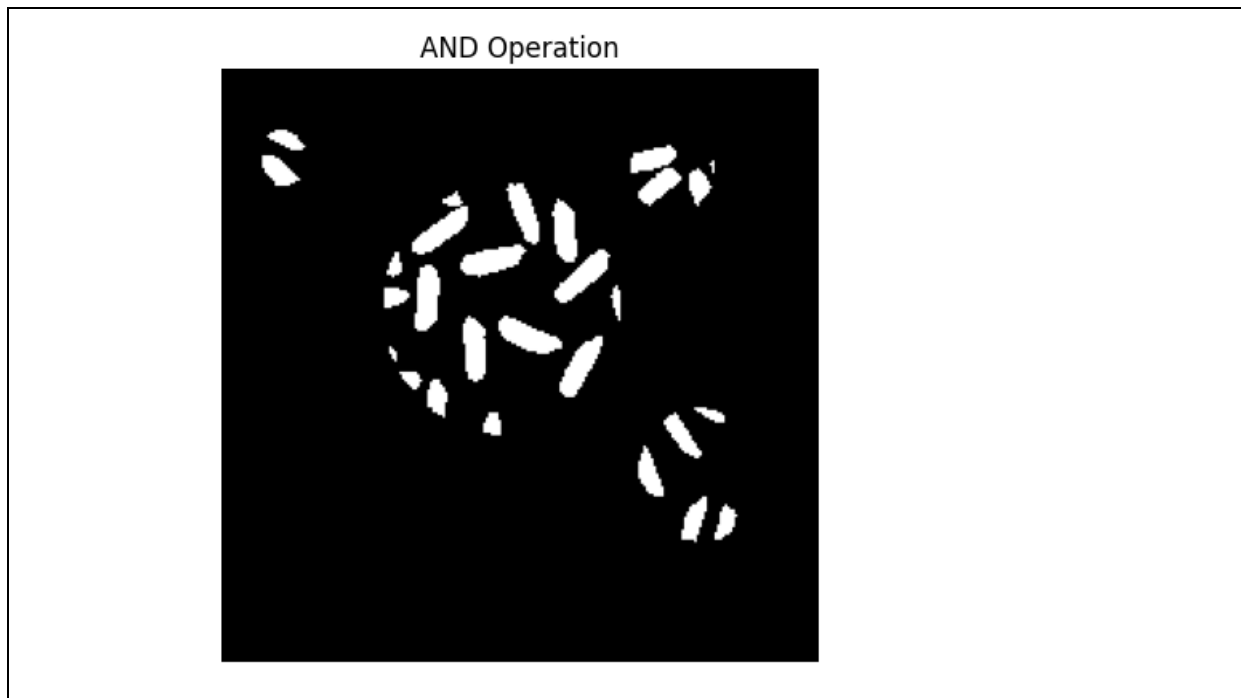
Explanation:

The output shows Binary Fig-1 and Binary Fig-2, obtained using `cv2.threshold()` with a threshold value of 127. All pixels above 127 are converted to white (255) and all pixels below are converted to black (0). Binary Fig-1 shows the circular shapes as solid white regions on a black background, while Binary Fig-2 shows the grain shapes as white regions on a black background. This confirms that both images were successfully converted into pure black-and-white (binary) form for further logical operations.

1. AND

```
and_result = cv2.bitwise_and(fig1_bin, fig2_bin)
plt.imshow(and_result, cmap='gray')
plt.title("AND Operation")
plt.axis("off")
plt.show()
```

Output:



Explanation:

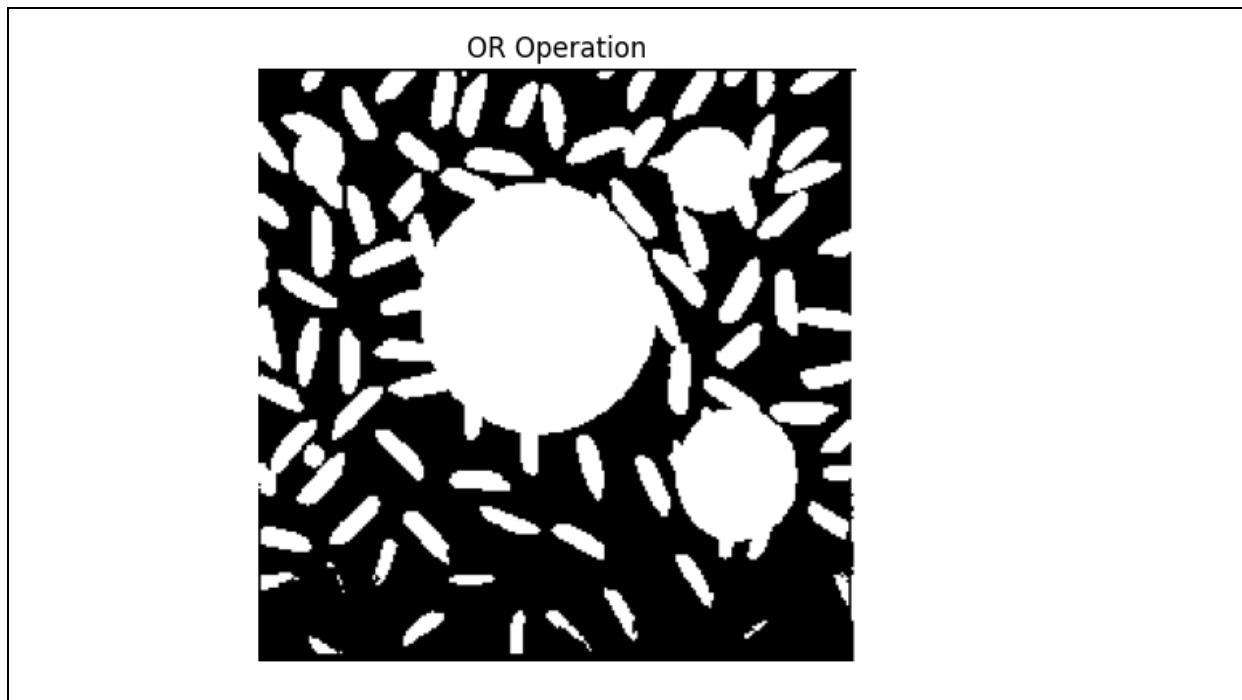
The output shows the result of `cv2.bitwise_and(fig1_bin, fig2_bin)`, which keeps only the pixels that are white in both binary images and turns everything else black. The result shows only the grain shapes that fall inside the circular region of `fig1`, since that's the only area where both images overlap as white. This confirms that the AND operation correctly extracted the common (overlapping) white regions between the two binary images.

2. OR

Code:

```
or_result = cv2.bitwise_or(fig1_bin, fig2_bin)
plt.imshow(or_result, cmap='gray')
plt.title("OR Operation")
plt.axis("off")
plt.show()
```

Output:



Explanation:

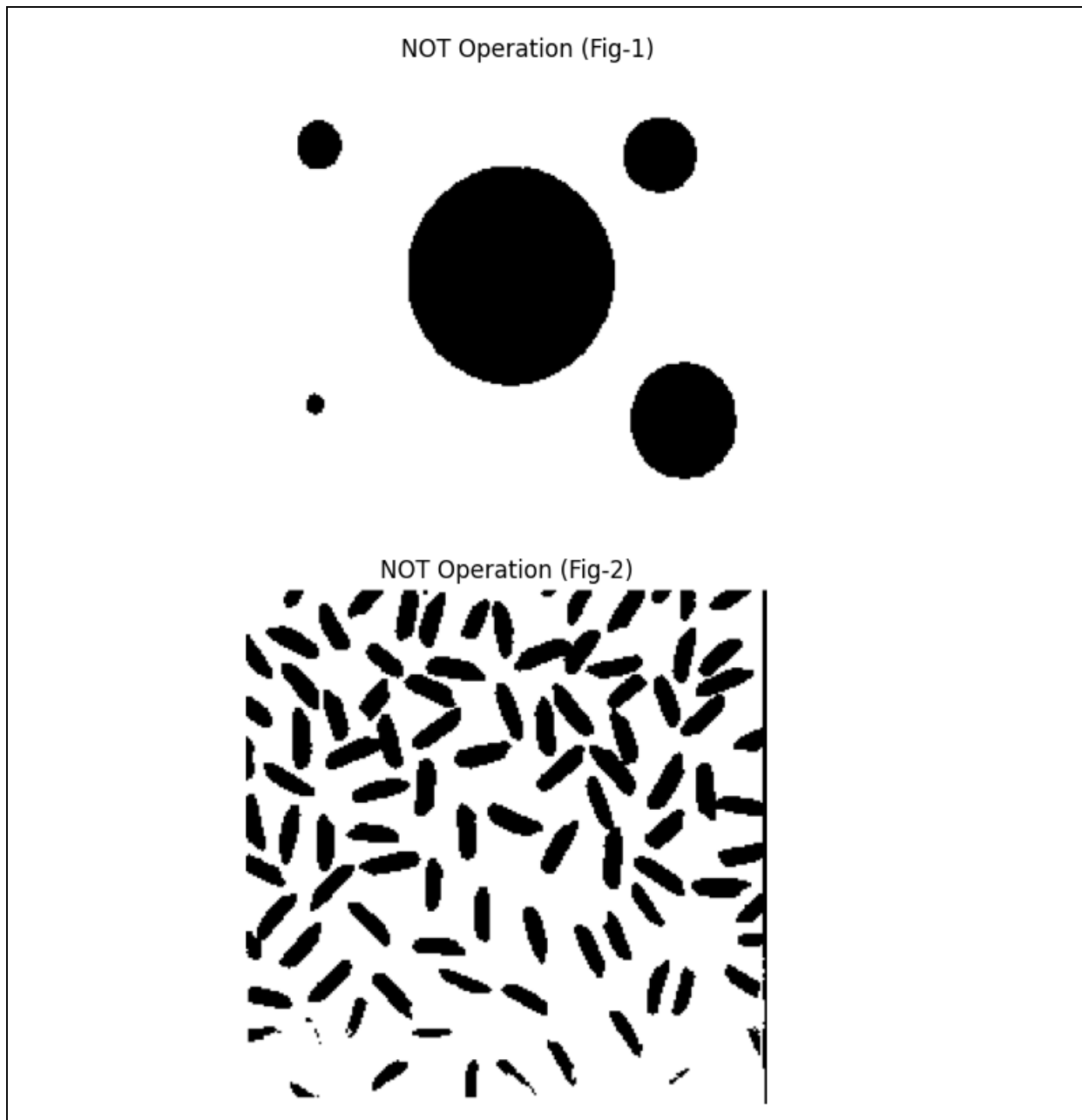
The output shows the result of `cv2.bitwise_or(fig1_bin, fig2_bin)`, which keeps a pixel white if it is white in either binary image (or both). The result shows both the circular shapes from `fig1` and the grain shapes from `fig2` combined together into one image. This confirms that the OR operation correctly merged all white regions from both binary images.

3. NOT

Code:

```
not_result = cv2.bitwise_not(fig1_bin)
plt.imshow(not_result, cmap='gray')
plt.title("NOT Operation (Fig-1)")
plt.axis("off")
plt.show()
not_result = cv2.bitwise_not(fig2_bin)
plt.imshow(not_result, cmap='gray')
plt.title("NOT Operation (Fig-2)")
plt.axis("off")
plt.show()
```

Output:



Explanation:

The output shows the result of `cv2.bitwise_not()` applied separately on `fig1_bin` and `fig2_bin`, which inverts every pixel white becomes black and black becomes white. In NOT Operation (Fig-1), the circles that were white are now black, and the previously black background is now white. Similarly, in NOT Operation (Fig-2), the grain shapes that were white are now black, with the background turned white. This confirms that the NOT operation correctly inverted the binary images

